

Centralizing ShopSphere Microservices using Spring Cloud Infrastructure and Shared Security

Business Scenario

The ShopSphere platform has successfully evolved into a distributed microservices ecosystem consisting of independent Spring Boot services such as the **Order Service**, **Inventory Service**, and **Notification Service**. Each service owns its own business data, communicates with other services through synchronous and asynchronous messaging, and can be developed and deployed independently.

As additional business services are introduced, the platform has begun facing several operational challenges common in large-scale microservice architectures.

Each microservice currently maintains its own application configuration, requiring configuration changes to be duplicated across multiple projects. Managing different configurations for development, testing, and production environments has become increasingly difficult and error-prone.

Furthermore, client applications must communicate directly with individual microservices by knowing their respective host names and port numbers. Whenever a service is relocated, scaled, or deployed to a different environment, client applications and dependent services must be updated with the new service locations.

The security implementation also exists independently within each microservice. JWT validation, authentication filters, security configuration, and authorization logic are duplicated across services, resulting in unnecessary code duplication and making it difficult to enforce consistent security policies across the platform.

To improve maintainability, scalability, and operational efficiency, the architecture team decides to introduce a centralized Spring Cloud infrastructure for the ShopSphere platform.

A **Spring Cloud Config Server** will be introduced to externalize and centrally manage the configuration of all microservices. Each service should retrieve its configuration from the Config Server during startup instead of maintaining local configuration files.

A **Service Discovery Server (Eureka)** will allow all microservices to register themselves dynamically. Rather than communicating using hardcoded host names and ports, services will be identified using logical service names registered with the discovery server.

An **API Gateway** will become the single entry point for all client applications. External consumers should no longer access individual microservices directly. Instead, every request should pass through the gateway, which will route requests to the appropriate backend service based on configured routes.

To eliminate duplicated security implementation, the engineering team will develop a **Common Security Module** that encapsulates reusable JWT authentication components,

security configuration, authentication filters, utility classes, and authorization logic. Every microservice requiring authentication should depend on this shared module instead of maintaining its own security implementation.

The API Gateway will be responsible for validating incoming JWT tokens before forwarding requests to backend services. Authentication and authorization should continue to be enforced using **Role-Based Access Control (RBAC)** and **Authority-Based Access Control**, ensuring that only authenticated users with appropriate permissions can access protected business resources.

The enhanced ShopSphere platform should demonstrate a production-ready microservices architecture where configuration is centrally managed, services are dynamically discovered, all external requests are routed through a centralized gateway, and security is implemented consistently using a reusable shared module.

TODOs

Spring Cloud Config Server

Centralized Configuration

- Create a Spring Cloud Config Server.
- Externalize configuration for all ShopSphere services.
- Store service configuration in a centralized configuration repository.
- Remove duplicated configuration from individual microservices.

Config Client Integration

Configure each microservice as a Config Client.

Services include:

- Order Service
- Inventory Service
- Notification Service
- API Gateway

Each service should:

- Retrieve configuration during startup.
- Validate successful connection to the Config Server.

- Support environment-specific configuration.
-

Service Discovery using Eureka

Eureka Server

- Configure a Eureka Discovery Server.
- Start the Discovery Server independently.

Service Registration

Register the following services with Eureka:

- API Gateway
- Order Service
- Inventory Service
- Notification Service

Each service should:

- Register automatically during startup.
- Be discoverable using its logical service name.
- Maintain its registration with periodic heartbeats.

Service Discovery

- Verify all services appear in the Eureka Dashboard.
 - Replace hardcoded service addresses with logical service identifiers.
 - Ensure services are dynamically discoverable.
-

API Gateway

Gateway Overview

- Understand the role of an API Gateway.
- Configure Spring Cloud Gateway.
- Define gateway routes for all business services.

Route Configuration

Configure routes for:

- Order Service
- Inventory Service
- Notification Service

Gateway Responsibilities

The gateway should:

- Act as the single entry point.
 - Route incoming requests.
 - Forward requests to appropriate services.
 - Centralize request processing.
 - Prevent direct external access to backend services.
-

Common Security Module

Shared Security Library

Create a reusable security module containing:

- JWT utility classes
- JWT validation logic
- Authentication filter
- Common security configuration
- Authentication entry point
- Authorization helper classes
- Shared constants
- Common exception handling components

Module Reusability

Integrate the shared module into:

- API Gateway
- Order Service
- Inventory Service
- Notification Service

Ensure security implementation is no longer duplicated across services.

JWT Authentication

Centralized Authentication

- Authenticate users using username and password.
 - Generate JWT after successful authentication.
 - Validate JWT for every protected request.
 - Extract authenticated user information from the token.
 - Populate the Spring Security Context.
-

Role-Based Access Control (RBAC)

Define business roles.

Examples:

- ADMIN
- MANAGER
- SUPPORT
- CUSTOMER

Assign business responsibilities to each role.

Restrict application functionality based on assigned roles.

Authority-Based Authorization

Define fine-grained permissions.

Examples:

- ORDER_READ
- ORDER_CREATE
- ORDER_UPDATE
- ORDER_DELETE
- INVENTORY_READ

- INVENTORY_UPDATE
- NOTIFICATION_READ
- NOTIFICATION_SEND

Map authorities to application roles.

Protect service-layer operations using method-level authorization.

Application Startup Sequence

Understand the startup order.

Config Server



Discovery Server (Eureka)



API Gateway



Order Service

Inventory Service

Notification Service

Request Flow

Understand the request lifecycle.

Client



API Gateway



JWT Authentication



Role & Authority Validation



Route Resolution



Service Discovery (Eureka)



Target Microservice



Business Processing



Response



Client

Application Validation

Validate the enhanced microservices platform by verifying:

- Config Server starts successfully.
 - All microservices load configuration from the Config Server.
 - Eureka Discovery Server registers all services.
 - Every service is visible in the Eureka Dashboard.
 - API Gateway correctly routes requests.
 - Backend services are not accessed directly by clients.
 - JWT authentication succeeds through the Gateway.
 - Invalid or expired JWT tokens are rejected.
 - Role-based authorization is enforced.
 - Authority-based authorization is enforced.
 - Method-level security continues to function correctly.
 - The Common Security Module is reused across all services.
 - Configuration changes are managed centrally without modifying individual services.
-

